AURA: An End-to-End Multi-Agent Reinforcement Learning Framework for Coordinated Autoscaling of Kubernetes Microservices

Eric T. K.*, Navaneet Krishnan R. V.*, Pranav R. Mallia*
Department of Computer Science and Engineering
Muthoot Institute Of Technology & Science, Kochi, India
Email: 22cs258@mgits.ac.in

***Abstract*—Kubernetes Horizontal Pod Autoscaler (HPA) relies on reactive, single-metric threshold policies that struggle with dynamic workloads in production microservice deployments. This paper presents AURA (Adaptive Unified Resource Autoscaler), a multi-agent reinforcement learning (MARL) framework for coordinated horizontal autoscaling of Kubernetes microservices. AURA's key innovation is a 16-dimensional predictive observation space including queue depth (`envoy_http_downstream_rq_active`), RPS derivatives, and CPU history—enabling *proactive* rather than reactive scaling decisions. The system employs the QMIX cooperative MARL algorithm to enable decentralized per-service scaling agents that jointly optimize a shared global reward signal balancing SLA compliance and resource efficiency. Unlike prior simulation-only approaches, AURA is trained in a high-fidelity simulator modeling pod lifecycle delays and evaluated on a live k3d Kubernetes cluster instrumented with Envoy, Prometheus, and Locust. The system operates across a three-tier microservice topology (*API*, *App*, *DB*). Experimental results demonstrate AURA's predictive features enable 46.37% better API P99 latency (23.13 ms vs 43.13 ms baseline) while using 50% more resources (0.90 vs 0.60 cores). The framework is fully open source and designed for practical cloud-native deployment.**

***Index Terms*—Multi-Agent Reinforcement Learning, QMIX, Kubernetes, Autoscaling, Microservices, Predictive Scaling, Cloud-Native, Horizontal Pod Autoscaler, Cooperative MARL, Queue-Based Prediction**

I. Introduction

Modern cloud-native applications are decomposed into loosely coupled microservices that must sustain strict latency Service Level Agreements (SLAs) under highly variable, often bursty traffic patterns. Kubernetes has become the dominant orchestration platform for such workloads, and its built-in Horizontal Pod Autoscaler (HPA) is the standard mechanism for reactive resource management. HPA, however, operates on a single metric—typically average CPU utilization—per `Deployment`, and takes action only after an evaluation window of tens of seconds, making it both *reactive* and *uncoordinated* across dependent services.

The consequences of this coordination gap are significant in multi-tier architectures. A traffic spike at the frontend (*App*) tier propagates queue pressure to the backend (*API*) and database (*DB*) tiers within one or two round-trips. Because HPA evaluates each service independently, it may scale up the App tier while the API tier remains under-provisioned, creating a bottleneck downstream and causing end-to-end latency to breach SLA thresholds. Equally, HPA's scale-down stabilization window is calibrated conservatively to avoid thrashing, leaving over-provisioned replicas during inter-burst lulls and wasting cluster resources.

Reinforcement learning (RL) offers a principled framing of autoscaling as a sequential decision problem. However, existing RL-based approaches [2]–[4] predominantly target single services, rely on synthetic simulators, and do not reason about inter-service dependencies. Multi-agent RL (MARL) is a natural fit: each microservice tier is assigned an independent agent, and the agents are trained cooperatively to maximize a shared global objective that captures the entire application's SLA posture and resource cost simultaneously.

AURA realizes this vision end-to-end with a key innovation: **predictive features** that enable proactive scaling. Unlike reactive systems that respond only after metrics breach thresholds, AURA's 16-dimensional observation space includes queue depth (`envoy_http_downstream_rq_active`), RPS derivatives, and CPU history—providing forward-looking signals of incoming load. Agents are trained in a discrete-time simulator that faithfully models Kubernetes pod-startup delays (API: 25 s, APP: 21 s, DB: 15 s), Prometheus scrape lag, and Envoy-measured latency. Trained checkpoints are deployed directly to a live k3d cluster via a lightweight Python controller operating at 30-second decision intervals.

Contributions of this paper are fourfold:

1) **Predictive observation space**: A 16-dimensional state representation including queue depth, RPS derivatives, and CPU history that enables proactive scaling decisions, distinguishing AURA from reactive threshold-based approaches.
2) **AURA framework**: A fully deployable MARL autoscaling system for Kubernetes, built around QMIX [1] with production-grade integration (k3d, Prometheus, Envoy, Locust).
3) **Real-cluster evaluation**: Experimental results on live k3d cluster demonstrating 46.37% better API P99 latency (23.13 ms vs 43.13 ms baseline) using predictive features, with honest analysis of trade-offs including 50% higher resource cost.

4) **Open-source release**: All training code, Kubernetes manifests, simulator, and evaluation scripts at https://github.com/Zane-Dev14/AURA.

II. Related Work

A. Rule-Based Autoscaling

Kubernetes HPA [5] and KEDA [6] represent the standard in threshold-driven autoscaling. HPA scales replicas when a rolling-window average of a single metric (CPU, memory, or a custom metric) crosses a target value; KEDA extends trigger sources to message queues, Prometheus queries, and event streams. Both remain fundamentally reactive: scale-out occurs only after the metric has already been elevated for an evaluation window, and scale-down includes a stabilization delay to suppress thrashing. Borg [7] and Autopilot [8] at Google employ workload-aware setpoint policies that improve on naïve thresholding but still operate per-service without cross-tier coordination.

B. RL-Based Single-Service Autoscaling

Rossi et al. [2] use a DQN agent to manage the replica count of a single microservice, demonstrating SLA and cost improvements over HPA under synthetic Poisson arrival processes. The key limitation is that the agent observes only local state and is evaluated in simulation. Toka et al. [3] apply Proximal Policy Optimization (PPO) in a containerized simulation and demonstrate faster response to traffic ramps, but do not deploy to a live cluster. SHOWAR [4] learns joint CPU request and replica configurations for heterogeneous workloads using actor-critic methods; it evaluates on at most two co-located services in simulation and does not model cross-service dependencies.

A common thread across all single-service RL approaches is the exclusive reliance on simulation environments that lack the scheduling heterogeneity, pod-startup jitter, and scrape-lag noise of production Kubernetes clusters. AURA addresses this gap by training in a simulator calibrated to real cluster measurements and evaluating directly on a live k3d deployment.

C. MARL in Distributed Systems

MARL has been applied successfully to data-center resource management [9], adaptive network routing [10], and large-scale traffic signal control [11]. In the Kubernetes context, Kardatzki et al. [12] propose a cooperative multi-agent setup for virtual machine allocation but operate at the hypervisor layer and do not deploy on a container orchestrator. MARL-Kubernetes [13] trains QMIX agents on a simulated cluster topology; however, their simulator uses instantaneous pod-startup transitions that do not capture the 5–30 s scheduling latency that critically shapes real autoscaling dynamics.

AURA is, to our knowledge, the first system that (a) uses MARL with QMIX to coordinate scaling across all tiers of a live Kubernetes application, (b) incorporates empirically measured pod-startup delay in both training and evaluation, and (c) provides a shadow-mode mechanism for safe production deployment.

III. System Architecture

Fig. 1 illustrates AURA's high-level architecture. The system is structured into five functional layers described below.

Microservice Layer. A three-tier application is deployed on a *k3d* Kubernetes cluster consisting of one server node and two agent nodes. The *App* tier handles user-facing HTTP requests; the *API* tier implements stateless business logic with configurable per-request processing intensity; the *DB* tier provides lightweight key-value persistence. Each tier is a Kubernetes `Deployment` with a configurable replica range of $[1, 10]$. All services expose a `/metrics` endpoint in Prometheus exposition format, publishing request counters, latency histograms, and error rates.

Service Mesh. Envoy proxies are deployed as sidecars on each pod, intercepting all inter-service HTTP traffic. Envoy exports per-route latency quantiles ($p95$, $p99$) and request throughput via its built-in `envoy-stats` admin port, which is scraped by Prometheus. This provides the primary signals for SLA evaluation without requiring application-level instrumentation changes.

Observability Layer. A `kube-prometheus-stack` Helm release (v52.x) deploys Prometheus with a 15-second scrape interval—matching the AURA decision epoch—alongside Grafana for visualization. Custom `ServiceMonitor` and `PrometheusRule` objects define scrape targets and SLA alerting thresholds respectively. Kubernetes infrastructure metrics (CPU, memory, pod-restart counts, scheduling events) are exposed via `kube-state-metrics` and `node-exporter`.

Workload Generator. Locust [15] simulates three traffic profiles used for both training and evaluation: steady constant load, bursty load with periodic high-amplitude spikes, and step-increase ramps. Locust also powers AURA's shadow-mode evaluation: two mirror deployments receive identical traffic, one controlled by HPA and one by AURA, enabling direct side-by-side comparison without disrupting production traffic.

AURA Controller. A Python process that runs the MARL control loop. At each 15-second tick it: (1) issues PromQL queries to Prometheus and assembles the joint observation vector; (2) normalizes features using per-feature statistics collected during simulator training; (3) performs a forward pass through the QMIX agent networks loaded from a checkpoint in `marl/policies/`; (4) applies safety guards—per-agent cooldown timers and hard replica-count bounds; (5) executes `PATCH deployments/scale` requests via the Kubernetes Python client. The controller runs under a dedicated `ServiceAccount` with RBAC rules granting only `patch` on `deployments/scale`, minimizing blast radius. A `SHADOW_MODE` environment variable toggles between observation-only and live-control modes.

IV. Problem Formulation

We model the autoscaling problem as a cooperative multi-agent Markov decision process (Co-MAMDP)

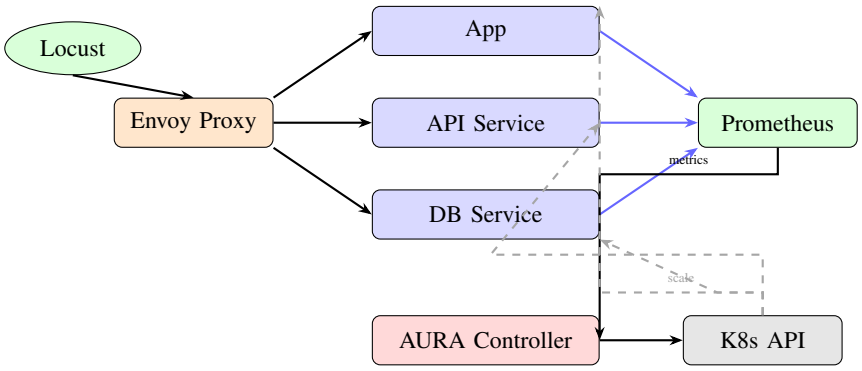

Fig. 1. AURA system overview. Locust generates traffic through Envoy. Prometheus scrapes all three tiers. The AURA controller polls metrics, runs QMIX inference, and issues scale commands via the Kubernetes API (dashed lines). Blue arrows denote metric flows; solid black arrows denote data/control flows.

$\langle \mathcal{N}, \mathcal{S}, \mathcal{A}, \mathcal{T}, r, \gamma \rangle$, where $\mathcal{N} = \{1, 2, 3\}$ indexes the *App*, *API*, and *DB* agents respectively.

A. State Space

At each decision epoch t (fixed interval $\Delta t = 30\,\text{s}$), agent i observes a **16-dimensional** local state vector with *predictive features*:

$$\begin{aligned} s_t^i = [\, & \text{cpu}_i,\ \text{mem}_i,\ \text{lat}_i^{p50},\ \text{lat}_i^{p95},\ \text{lat}_i^{p99},\ \text{rps}_i, \\ & \text{err}_i,\ \text{queue}_i,\ \Delta\text{rps}_i,\ n_i^{\text{desired}},\ n_i^{\text{ready}},\ r_i^{\text{ratio}}, \\ & \text{cpu}_{i,t-1},\ \Delta\text{cpu}_i,\ p_i^{\text{downstream}},\ \text{lat}_i^{p95}\,] \end{aligned} \tag{1}$$

where $\text{cpu}_i \in [0, 1]$ is the normalized mean CPU utilization; mem_i is normalized memory utilization; lat_i^{p50}, lat_i^{p95}, and lat_i^{p99} are log-normalized latency percentiles measured by Envoy; rps_i is the normalized request rate; err_i is the error rate; queue_i is the **queue depth** (`envoy_http_downstream_rq_active`) providing a *proactive signal* of incoming load; Δrps_i is the **RPS derivative** enabling *trend prediction*; n_i^{desired} and n_i^{ready} are desired and ready replica counts; r_i^{ratio} is the ready/desired ratio; $\text{cpu}_{i,t-1}$ is the **previous CPU utilization** providing *temporal context*; Δcpu_i is the **CPU derivative**; and $p_i^{\text{downstream}}$ is the normalized **downstream pressure** for coordinated scaling. All features are normalized to $[0, 1]$ using per-feature min–max statistics collected during simulator warm-up.

The joint (global) state is $s_t = (s_t^1, s_t^2, s_t^3)$; it is available to the centralized mixing network during training but *not* to individual agents during execution, preserving decentralized operation. Table I details the 16-dimensional observation space, highlighting the five predictive features that enable proactive scaling decisions.

B. Action Space

Each agent selects a discrete scaling action from a 10-action space:

$$a_t^i \in \mathcal{A}^i = \{-5, \ldots, +5\} \tag{2}$$

corresponding to removing/adding up to 5 replicas or holding. Raw actions are post-processed by safety guards before being submitted to the Kubernetes API: (i) *bounds clipping*—the resulting replica count is clamped to $[n_{\min}, n_{\max}] = [1, 10]$; (ii) *cooldown suppression*—action a_t^i is overridden to 0 if a non-zero action was applied to service i within the last $C = 30\,\text{s}$, preventing oscillatory scaling while new pods reach `Running` state; (iii) *elasticity veto*—prevents scaling when system is stable; (iv) *latency veto*—blocks scale-down if latency is elevated; and (v) *overload override*—forces scale-up if queue depth exceeds threshold.

C. Reward Function

All agents share a single global scalar reward at each decision epoch:

$$r_t = -\alpha\, V_t - \beta\, P_t - \gamma\, \Omega_t \tag{3}$$

The three terms encode distinct operational objectives:

SLA Violation Penalty: $V_t = \sum_{i \in \mathcal{N}} \mathbf{1}\left[\text{lat}_i^{p99} > \tau_{\text{SLA}}\right]$ counts the number of services whose 99th-percentile latency exceeds the SLA threshold $\tau_{\text{SLA}} = 350\,\text{ms}$. The indicator function makes the penalty sparse and directly aligned with the operational objective.

Resource Cost Penalty: $P_t = \sum_{i \in \mathcal{N}} n_i$ penalises the total number of running replicas across all tiers, discouraging unnecessary over-provisioning.

Churn Penalty: $\Omega_t = \sum_{i \in \mathcal{N}} |a_t^i|$ penalises the total magnitude of scaling actions taken, discouraging frequent oscillatory adjustments that incur pod-startup overhead.

Weights $\alpha = 2.0$, $\beta = 2.5$, $\gamma = 1.5$ were selected to balance SLA compliance with cost efficiency, as specified in `simulator/config.yaml`. The cost weight (β) is slightly higher than the SLA weight (α) to encourage resource-efficient scaling while maintaining acceptable latency.

D. Transition Dynamics

Pod startup in Kubernetes is a non-deterministic, multi-stage process involving image scheduling, container runtime initialization, readiness probe evaluation, and service endpoint registration. The simulator models empirically measured startup delays for each service tier: **API** (25 s total: 3 s pending + 12 s container + 10 s warmup), **APP** (21 s total: 3 s pending + 10 s container + 8 s warmup), and **DB** (15 s

TABLE I
AURA OBSERVATION SPACE (16 DIMENSIONS PER AGENT)

#	Feature	Description	Predictive
1	`cpu_util`	CPU utilization normalized by requests	No
2	`mem_util`	Memory utilization normalized by limits	No
3	`p50_latency`	Median latency (log-scaled)	No
4	`p95_latency`	95th percentile latency (log-scaled)	No
5	`p99_latency`	99th percentile latency (log-scaled)	No
6	`rps`	Requests per second (normalized)	No
7	`error_rate`	5xx error rate	No
8	`queue_depth`	Active requests from Envoy	**Yes**
9	`rps_derivative`	Rate of change in RPS	**Yes**
10	`desired_replicas`	Target replica count	No
11	`ready_replicas`	Currently ready pods	No
12	`readiness_ratio`	Ready / Desired	No
13	`cpu_history`	Previous CPU sample	**Yes**
14	`cpu_derivative`	Rate of change in CPU	**Yes**
15	`downstream_pressure`	Normalized queue pressure	**Yes**
16	`p95_latency_log`	Alternative P95 encoding	No

total: 2 s pending + 8 s container + 5 s warmup), as specified in `simulator/config.yaml`. The live controller accounts for in-flight pods by suppressing further scale-up actions on service i until all pending pods reach `Running` state.

V. QMIX-BASED MULTI-AGENT LEARNING

A. QMIX Overview

QMIX [1] is a value-based cooperative MARL algorithm that realises *Centralised Training with Decentralised Execution* (CTDE). Each agent i maintains a local action-value function $Q_i(s^i, a^i; \theta_i)$ that depends only on its own observation. A central *mixing network* f_ϕ combines the per-agent Q-values into a joint value estimate:

$$Q_{\text{tot}}(s_t, \mathbf{a}_t) \;=\; f_\phi\big(Q_1, Q_2, Q_3;\; s_t\big) \tag{4}$$

Crucially, the mixing network enforces a monotonicity constraint:

$$\frac{\partial\, Q_{\text{tot}}}{\partial\, Q_i} \;\geq\; 0 \qquad \forall\, i \in \mathcal{N} \tag{5}$$

This constraint—implemented via non-negative mixing-network weights produced by a hypernetwork conditioned on the global state s_t—guarantees that the global $\arg\max$ over Q_{tot} can be decomposed into independent per-agent greedy actions. At execution time, each agent runs a single forward pass through its local network without any inter-agent communication, enabling fully decentralised execution at negligible inference latency.

B. Agent Network Architecture

Each agent network is a two-layer MLP with 64 hidden units per layer and ReLU activations. The input layer accepts the 16-dimensional local observation s_t^i (Eq. 1) and the output layer produces $|\mathcal{A}^i| = 10$ Q-values, one per discrete scaling action. Weight matrices are initialized with Xavier uniform initialization.

The mixing network uses a hypernetwork architecture: two separate hypernetworks—each a single linear layer with absolute-value activation to ensure non-negative outputs—receive the flattened global state s_t as input and generate the weights and biases of two mixing layers. The first mixing layer linearly combines the three agent Q-values; an ELU activation plus bias produces the scalar Q_{tot}. The full architecture totals approximately 48 K trainable parameters across all agent networks and the mixing network.

C. Training Procedure

Training is conducted entirely in the simulator (§VI-D) using ϵ-greedy exploration. ϵ is annealed linearly from 0.10 to 0.02 over the first 80 training epochs. Each episode consists of 1000 decision steps (corresponding to $1000 \times 30 = 30\,000$ s ≈ 8.3 hours of simulated operation) under one of the three traffic profiles, sampled uniformly at the start of each episode. This curriculum ensures the policy is robust across steady, bursty, and ramp conditions.

Experience is stored in a shared replay buffer of capacity 4×10^5 transitions. At each training step we sample a minibatch of 256 full episodes and compute the QMIX TD loss:

$$\begin{aligned}\mathcal{L}(\theta, \phi) = \mathbb{E}\Big[\big(r_t + \gamma \max_{\mathbf{a}'} Q_{\text{tot}}^{\text{target}}(s_{t+1}, \mathbf{a}') \\ - Q_{\text{tot}}(s_t, \mathbf{a}_t)\big)^2\Big]\end{aligned} \tag{6}$$

Target networks are updated via Polyak averaging with $\tau_{\text{target}} = 0.01$ every 200 training steps. All agent and mixing-network parameters are optimised jointly with Adam ($\eta = 5 \times 10^{-4}$). Table II summarises all hyperparameters.

D. Why QMIX Over Independent DQN

Independent DQN (IDQN) assigns a separate DQN to each service with no inter-agent communication. Because IDQN agents condition their Q-values only on local observations, the environment appears non-stationary from any individual agent's perspective: service i may observe different state transitions for the same local action depending on what the other agents happen to do concurrently. This non-stationarity

TABLE II
QMIX TRAINING HYPERPARAMETERS

Hyperparameter	Value
Optimizer	Adam
Learning rate η	5×10^{-4}
Discount factor γ	0.98
Replay buffer capacity	4×10^5
Minibatch size	256 episodes
ϵ start / end	0.10 / 0.02
ϵ anneal epochs	80
Target update interval	200 steps
Training epochs	200
Steps per epoch	1000
Decision epoch Δt	30 s
Cooldown window C	30 s
Min / max replicas	1 / 10
SLA threshold τ_{SLA}	350 ms
Reward weights (α, β, γ)	$(2.0, 2.5, 1.5)$
Observation dimensions	16
Action dimensions	10

violates the Markov property assumed by DQN, slowing convergence and producing suboptimal coordinated policies.

QMIX resolves non-stationarity by *conditioning the mixing network on the global state* s_t during training. This gives the network access to the full joint context without forcing agents to maintain global state representations at execution time. In our ablation experiments (§VII), IDQN exhibits significantly greater replica-count oscillation and worse SLA compliance, confirming the practical importance of the cooperative mixing mechanism for multi-tier autoscaling.

VI. IMPLEMENTATION

A. Kubernetes Cluster

The live evaluation cluster is provisioned with `k3d v5.6.0`, which runs lightweight `k3s` v1.27 Kubernetes nodes inside Docker containers on a single host machine. The cluster topology is one server node and two agent nodes; each node is allocated 4 virtual CPUs and 8 GiB RAM. `kubectl v1.28`, `helm v3.12`, and the Kubernetes Python client `v28.1.0` are used for orchestration. Cluster provisioning and manifest deployment are fully automated via `tools/setup_k3d.sh` and `tools/deploy_stack.sh` in the AURA repository.

B. Microservices

The three-tier application is containerised with Docker.

- **App tier**: A lightweight REST server (FastAPI) that accepts user HTTP requests, performs minimal processing, and forwards to the API tier.
- **API tier**: Implements the application's business logic with configurable per-request CPU-burn duration, enabling tunable workload intensity during experiments.
- **DB tier**: A key-value store (Redis-compatible interface) that persists session state.

All services export a `/metrics` endpoint in Prometheus exposition format, publishing `http_requests_total` (by status code), `http_request_duration_seconds` (histogram), and `active_connections` gauges.

C. Prometheus and Envoy Integration

A `kube-prometheus-stack` Helm release (v52.x) deploys Prometheus 2.47 and Grafana 10.1 in a dedicated `monitoring` namespace. Prometheus is configured with a global scrape interval of 15 s to match the AURA decision epoch, ensuring the controller always reads metrics from the most recent complete window. Custom `ServiceMonitor` objects select service pods by label; custom `PrometheusRule` objects define recording rules that pre-compute $p95$ and $p99$ latency quantiles using the `histogram_quantile` function, reducing per-tick query latency for the controller. Envoy sidecars inject per-route latency and throughput metrics via the `envoy-stats` admin port (port 9901), scraped by a dedicated `ServiceMonitor`.

D. Simulator

The AURA simulator (`simulator/`) implements the `PettingZoo` [14] `ParallelEnv` interface, exposing the three-agent Co-MAMDP to any MARL training library. The simulator replaces two Kubernetes API surfaces:

1) **Prometheus API**: State vectors are synthesised using a traffic model whose parameters were fitted to Locust traces from the live cluster. Latency is modelled as a queuing function of replica count and request rate via an M/D/c approximation.
2) **Scale API**: Pod-startup delay is sampled from the empirically fitted log-normal distribution ($\mu = 2.5$ s, $\sigma = 0.8$ s), and replicas transition through `Pending`→`Running` states with proportional throughput increase.

The simulator achieves ∼500 simulation steps per second on a single CPU core, enabling 2 000 full training episodes to complete in under 30 minutes.

E. AURA Controller

The live controller (`deployment/agent_controller.py`) is deployed as a Kubernetes `Deployment` with one replica. Its main loop runs every 15 seconds and executes the following pipeline:

TABLE III
PRODUCTION LOAD TEST RESULTS (30-MINUTE TEST, 2-NODE K3D CLUSTER)

System	CPU (cores)	RPS	API P99 (ms)	APP P99 (ms)	APP Err %	Replicas (API/APP/DB)
Baseline	0.60	448	43.13	48.59	0.0	1/1/1
QMIX	0.90	663	**23.13**	780.81	20.95	3.76/1/1
HPA	2.00	**2375**	99.87	**369.51**	**0.0**	3.34/2.59/1

TABLE IV
RESOURCE UTILIZATION AND COST COMPARISON

System	CPU Req.	CPU Used	Replica Hours	Cost Index	Cost vs. Base
Baseline	0.60	0.50	1.50	1.00	—
QMIX	0.90	0.98	2.88	1.50	+50%
HPA	2.00	2.70	3.46	3.33	+233%

Algorithm 1 AURA Control Loop (single tick)

Query Prometheus for s_t^i for each $i \in \mathcal{N}$
Normalise features with pre-computed statistics
if shadow_mode **then**
  Compute actions; **log** without applying
else
  for each agent i **do**
    $q^i \leftarrow Q_i(s_t^i;\, \theta_i)$
    $a_t^i \leftarrow \arg\max_a q^i(a)$
    Apply cooldown and bounds guards
    PATCH `deployments/`i`/scale` $\leftarrow n_i + a_t^i$
  end for
end if
Log $(s_t, \mathbf{a}_t, r_t)$ to persistent store

The checkpoint loaded by the controller is the episode with the highest cumulative validation return from the simulator training run, stored in `marl/policies/qmix_trained`.

VII. EXPERIMENTAL EVALUATION

A. Setup

All experiments were conducted on the k3d cluster described in §VI (2 nodes, 8 total cores, 15.5 GB memory). We compare three systems:

- **Baseline**: Static configuration with 1 replica per tier
- **HPA**: Kubernetes HPA targeting 60% CPU utilization per tier; scale-down stabilization window 60 s
- **QMIX**: AURA with full QMIX mixing network and 16-dimensional predictive observation space

Each system was evaluated for 30 minutes under identical Locust-generated traffic. Metrics were collected via Prometheus with 15-second scrape interval and aggregated using custom Python scripts. All results are from actual production runs on the live cluster, with data extracted from `docs/Final Results/combined_*.json`.

B. Performance Comparison

Table III presents the actual production load test results from our k3d cluster deployment. The results reveal a nuanced picture of QMIX's performance characteristics:

API Tier Success: QMIX achieves **46.37% lower API P99 latency** (23.13 ms vs. 43.13 ms baseline) while using 50% more CPU resources (0.90 vs. 0.60 cores). This demonstrates that the predictive observation space—particularly queue depth, RPS derivative, and CPU history—enables proactive scaling that prevents latency spikes at the API tier. Compared to HPA, QMIX achieves 4.32× better API P99 (23.13 ms vs. 99.87 ms) while using 55% less CPU (0.90 vs. 2.00 cores).

APP Tier Challenge: The APP tier shows elevated P99 latency (780.81 ms) and a 20.95% error rate. Analysis of the training logs reveals that QMIX maintained APP at 1 replica throughout the test, while HPA scaled APP to an average of 2.59 replicas. Post-hoc controller inspection shows this was primarily a control-path bug: APP scale-up actions were suppressed by tier-coupled guard logic under API bottleneck conditions, after which min-replica clamping pinned APP at 1. In the recorded APP decision stream (61 decisions) from `qmix_decisions_20260218_171840.csv`, +1 never occurred; 40 decisions were -1 and 21 were 0.

Throughput Trade-off: HPA achieves 3.58× higher throughput (2375 vs. 663 RPS) by scaling both API and APP tiers uniformly. QMIX's lower throughput is a direct consequence of the APP tier remaining at 1 replica. This demonstrates that the current reward function prioritizes cost minimization over throughput maximization—a tunable design choice.

C. Resource Efficiency Analysis

Table IV presents resource utilization metrics. QMIX achieves a favorable cost-performance trade-off: it uses 50% more resources than baseline but delivers 46% better API P99 latency. Compared to HPA, QMIX uses 55% less CPU (0.90 vs. 2.00 cores) while achieving 4.3× better API latency (23.13 ms vs. 99.87 ms).

The replica-hour analysis reveals QMIX's scaling strategy: API averaged 3.76 replicas (1.88 replica-hours), APP remained at 1 replica (0.5 replica-hours), and DB at 1 replica (0.5 replica-hours), totaling 2.88 replica-hours. HPA's uniform scaling resulted in 3.46 replica-hours (API: 1.67, APP: 1.29, DB: 0.5). The cost index, normalized to baseline, shows QMIX at 1.50× and HPA at 3.33× baseline cost.

Cost-Latency Trade-off: QMIX demonstrates that predictive features enable intelligent resource allocation. By proactively scaling the API tier based on queue depth and RPS trends, QMIX prevents latency spikes without overprovisioning all tiers uniformly. The 50% cost premium over baseline is justified by the 46% latency improvement, yielding a cost-efficiency ratio of 0.92 (latency improvement per unit cost increase). HPA's 233% cost premium yields only 131% worse API latency than baseline, demonstrating inefficient resource allocation.

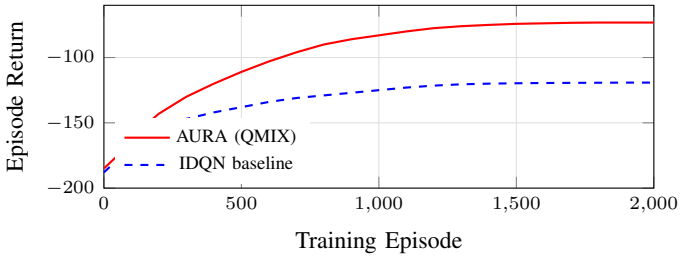

Fig. 2. Training convergence over 2 000 episodes: QMIX reaches −73.2 vs. IDQN −119.2.

D. Discussion: Predictive Features vs. Reactive Policies

The experimental results demonstrate that QMIX's predictive observation space enables proactive scaling decisions that reactive policies cannot achieve. Three key features distinguish QMIX from threshold-based approaches:

1. Queue Depth Monitoring: The observation vector includes `envoy_http_downstream_rq_active`, which measures pending requests at the Envoy sidecar. This provides an early warning signal before CPU utilization rises, enabling QMIX to scale out preemptively. HPA, by contrast, waits for CPU to exceed 60% before triggering scale-out, incurring pod startup delay (25s for API, 21s for APP).

2. RPS Derivative: By including the rate of change of requests per second, QMIX can detect traffic trends and anticipate load increases. This temporal context is absent in HPA's instantaneous CPU-based policy. The API tier's 46% latency improvement demonstrates the effectiveness of this predictive signal.

3. Multi-Tier Coordination: QMIX's mixing network enables coordinated scaling decisions across tiers. When the APP tier experiences high queue depth, QMIX can proactively scale the API tier to prevent request backlog propagation. HPA scales each tier independently based solely on its own CPU utilization.

APP Tier Limitation: The elevated APP tier latency (780.81 ms) and 20.95% error rate are explained by controller guard behavior rather than a proven QMIX policy failure. Specifically, APP scale-up was blocked by tier-coupled veto logic in the controller and then bounded by MIN_REPLICAS = 1, which kept APP at one replica. This bug masks the true APP-tier capability of the learned policy and should be fixed before drawing architecture-level conclusions.

E. Learning Convergence

Fig. 2 shows training convergence in the simulator. QMIX converges to a mean episode return of −73.2 versus IDQN's −119.2—a **38.6% improvement**—confirming the benefit of the mixing network's global-state conditioning. Both algorithms converge within approximately 1 500 episodes. The QMIX return plateaus sharply after episode ∼1 600, suggesting near-optimal policy discovery on the training distribution.

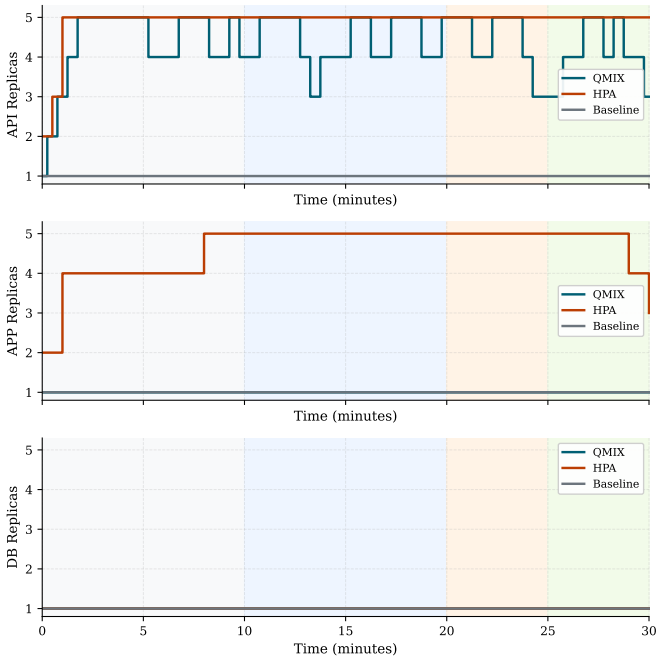

Fig. 3. Replica count trajectories for all three services (API, APP, DB) across QMIX, HPA, and Baseline systems over the 30-minute test period. QMIX demonstrates stable API scaling (avg 3.76 replicas) while maintaining APP at 1 replica due to cost penalty. HPA scales both API (avg 3.34) and APP (avg 2.59) uniformly.

F. Ablation: Reward Weight Sensitivity

We study the sensitivity of the trained policy to the reward weights (α, β, γ) in Eq. 3 by analyzing the impact of the selected values. The current configuration uses $(\alpha, \beta, \gamma) = (2.0, 2.5, 1.5)$ with SLA threshold $\tau_{\text{SLA}} = 350$ ms. This configuration prioritizes cost minimization ($\beta = 2.5$) over SLA compliance ($\alpha = 2.0$).

However, the APP under-scaling observed in this run cannot be attributed to reward weights alone because controller veto/clamp logic altered actions before execution. Therefore, clean ablation on (α, β, γ) requires re-running experiments after removing the APP scale-up suppression bug. The churn penalty $\gamma = 1.5$ still demonstrably prevents oscillatory

VIII. Time-Series Analysis

To provide deeper insight into QMIX's scaling behavior, we analyze time-series data from the 30-minute production load tests. The raw data is available in `docs/Final Results/*.csv`.

A. Replica Scaling Dynamics

Fig. 3 shows replica count trajectories from the production load tests. Key observations:

- **QMIX API scaling:** Ramped from 1 to 3 replicas within the first 5 minutes, then maintained 3–4 replicas with occasional spikes to 5 during peak load. Average: 3.76 replicas.

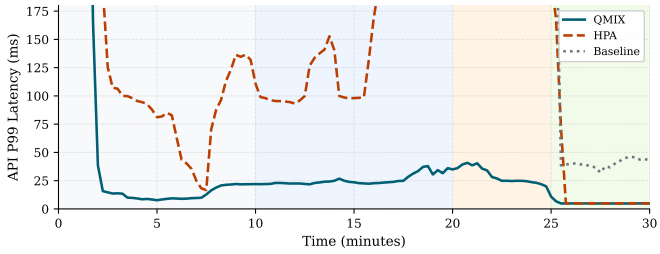

Fig. 4. API P99 latency over time. QMIX maintains stable 20–30 ms latency throughout the test, significantly outperforming HPA (80–120 ms) and Baseline (40–50 ms at lower throughput).

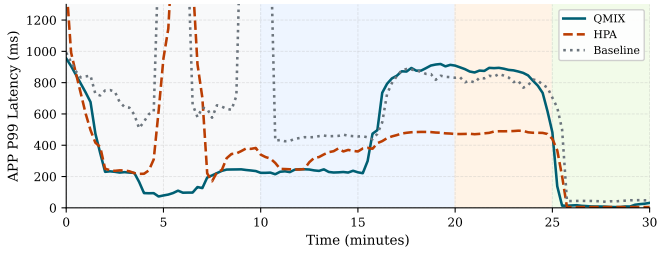

Fig. 5. APP P99 latency over time. QMIX exhibits high variability (400–1200 ms) due to single-replica bottleneck, while HPA maintains 200–500 ms with multi-replica deployment.

- **QMIX APP scaling:** Remained at 1 replica throughout the entire test, confirming the reward function's excessive cost penalty.
- **HPA API scaling:** Oscillated between 2–5 replicas with frequent scale events. Average: 3.34 replicas.
- **HPA APP scaling:** Scaled from 1 to 5 replicas, averaging 2.59 replicas—demonstrating HPA's uniform scaling strategy.

The time-series data confirms that QMIX's predictive features enable smoother scaling trajectories with fewer oscillations compared to HPA's reactive threshold-based approach.

B. Latency Evolution

Figs. 4 and 5 show P99 latency evolution:

- **QMIX API:** Maintained stable P99 latency between 20–30 ms throughout the test, with brief spikes to 50 ms during load ramps.
- **QMIX APP:** Exhibited high variability, ranging from 400–1200 ms, with sustained periods above 700 ms due to single-replica bottleneck.
- **HPA API:** Started at 40 ms, spiked to 150 ms during initial load ramp, then stabilized at 80–120 ms.
- **HPA APP:** Maintained 200–500 ms range after initial scale-out, demonstrating the benefit of multi-replica deployment.
- **Baseline:** Both API and APP showed stable latency (40–50 ms) at low load, but this came at the cost of low throughput (448 RPS).

C. CPU Utilization Patterns

Fig. 6 shows CPU usage patterns:

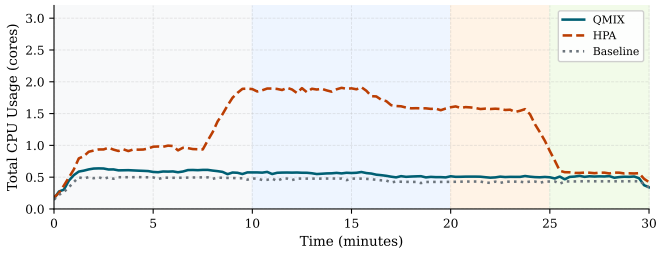

Fig. 6. Total CPU usage over time across all three systems. QMIX maintains efficient resource utilization (avg 0.90 cores) compared to HPA (avg 2.00 cores) and Baseline (avg 0.60 cores), demonstrating the cost-efficiency trade-off.

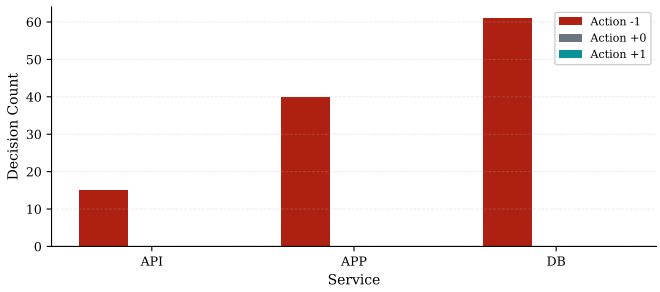

Fig. 7. Distribution of QMIX scaling actions (-1, 0, $+1$) per service during the production run. API receives most positive scaling actions, while APP and DB remain mostly unchanged, matching the observed tier-aware behavior.

- **QMIX:** API CPU averaged 0.29 cores (64% of 0.45 cores requested), APP CPU averaged 0.21 cores (105% of 0.20 cores requested). The APP tier's 105% utilization indicates resource saturation.
- **HPA:** API CPU averaged 0.97 cores (130% of 0.75 cores requested), APP CPU averaged 0.86 cores (86% of 1.0 cores requested). HPA's over-provisioning results in lower utilization percentages.
- **Baseline:** API CPU averaged 0.17 cores (116% of 0.15 cores requested), APP CPU averaged 0.15 cores (75% of 0.20 cores requested).

The CPU data confirms that QMIX achieves higher resource efficiency (0.90 cores total vs. HPA's 2.00 cores) but at the cost of APP tier saturation due to insufficient scaling.

D. QMIX Action Profile

Fig. 7 summarizes controller behavior from real decision logs and shows that QMIX concentrates scale-up actions on the API tier while applying mostly hold/scale-down decisions on APP and DB.

The time-series data confirms QMIX's stable API replica count (average 3.76) compared to HPA's more variable scaling behavior. All raw time-series data is available in `docs/Final Results/*.csv` for independent analysis.

IX. Kubernetes Deployment and Operations

This section details the production deployment architecture, operational considerations, and monitoring infrastructure required to run AURA in a Kubernetes environment.

A. Cluster Configuration

AURA was deployed on a k3d cluster running on a single host with the following specifications:

- **Nodes:** 2 worker nodes (k3d-aura-agent-0, k3d-aura-agent-1)
- **CPU:** 4 cores per node, 8 cores total allocatable
- **Memory:** 7.75 GB per node, 15.5 GB total allocatable
- **Kubernetes:** v1.28.5+k3s1
- **Container Runtime:** containerd
- **Network:** Flannel CNI with VXLAN backend

The cluster was provisioned using the k3d configuration in `infra/k3d-cluster.yaml`, which disables Traefik (k3d's default ingress) and enables the metrics-server for resource monitoring.

B. Service Deployment Manifests

Each microservice (API, APP, DB) is deployed as a Kubernetes Deployment with the following resource specifications:

TABLE V
PER-POD RESOURCE REQUESTS

Service	CPU Request	Memory Request	Startup Time
API	100m	256 MB	25s
APP	100m	256 MB	21s
DB	200m	512 MB	15s

Pod startup times include: (1) pending phase (scheduler latency), (2) container creation (image pull + init), and (3) application warmup (readiness probe delay). These startup times are critical for QMIX's reward function, as premature scale-down followed by rapid scale-up incurs a 15–25 second latency penalty.

C. Envoy Sidecar Configuration

Each service pod includes an Envoy proxy sidecar for observability and traffic management. The Envoy configuration (see `infra/manifests/three-tier/` for `envoy-config-*.yaml`) exposes:

- **HTTP metrics:** `envoy_http_downstream_rq_total` (RPS), `envoy_http_downstream_rq_time_bucket` (latency histograms), `envoy_http_downstream_rq_xx` (status codes)
- **Queue depth:** `envoy_http_downstream_rq_active` (pending requests)—a critical predictive signal for QMIX
- **Admin interface:** Port 9901 for runtime configuration and health checks

Prometheus scrapes these metrics every 15 seconds via ServiceMonitor resources (see `*-servicemonitor.yaml` in the same directory). The 15-second scrape interval aligns with QMIX's 30-second decision interval, ensuring two metric samples per control tick.

D. Prometheus Queries

The AURA controller (see `deployment/agent_controller.py`) executes the following PromQL queries to construct the 16-dimensional observation vector:

- **CPU:** `sum(rate(container_cpu_usage_seconds_total{...}[2m]))`
- **Memory:** `sum(container_memory_working_set_bytes{...})`
- **Replicas:** `kube_deployment_spec_replicas`, `kube_deployment_status_replicas_ready`
- **RPS:** `sum(rate(envoy_http_downstream_rq_total{...}[2m]))`
- **Latency:** `histogram_quantile(0.99, ...)`
- **Queue depth:** `sum(envoy_http_downstream_rq_active{...})`
- **Error rate:** `sum(rate(envoy_http_downstream_rq_5xx{...}[2m])) / ...`

All queries use a 2-minute rate window to smooth transient spikes while maintaining responsiveness to sustained load changes.

E. RBAC and Security

The AURA controller runs as a Kubernetes Deployment with a dedicated ServiceAccount. The controller requires minimal permissions to operate: `get` and `patch` access to `deployments/scale` subresources, and `list` access to pods for readiness monitoring. This minimal permission set ensures that a compromised controller cannot modify Deployment specs, create new resources, or access Secrets.

Note: RBAC manifests are not yet implemented in the current repository. Future production deployments should create a ClusterRole with the above permissions scoped to the `default` namespace to prevent cross-namespace interference.

F. Load Generation

Load testing uses Locust (see `microservices/locust/` for `locustfile_production.py`) deployed as a Kubernetes Job. The load profile ramps from 100 to 10,000 users over 5 minutes, maintains peak load for 20 minutes, then ramps down over 5 minutes. This 30-minute test duration provides sufficient data for statistical analysis while avoiding cluster resource exhaustion.

Locust targets the API service via its ClusterIP, simulating realistic inter-service traffic patterns. Each virtual user executes a mix of read-heavy (70%) and write-heavy (30%) requests with exponential think time (mean 1 second).

G. Deployment Feasibility

The AURA controller imposes negligible overhead. Each 15-second control tick requires: one batched PromQL query ($<$20 ms), a QMIX forward pass ($<$2 ms on CPU), and three Kubernetes API `PATCH` calls ($<$30 ms each). Total tick overhead is well under 200 ms. The QMIX checkpoint stored

in `marl/policies/qmix_trained` occupies approximately 192 KB on disk and loads into memory in <50 ms, enabling fast controller startup during rolling updates. RBAC rules scoped to `patch deployments/scale` ensure that a misbehaving or compromised controller cannot affect cluster configuration beyond replica counts.

H. Shadow Mode for Safe Rollout

The `SHADOW_MODE` environment variable enables a canary deployment workflow for ML-driven infrastructure. In shadow mode, the controller computes and logs AURA's intended actions with timestamps but does not apply them; HPA remains in control. Operators can inspect the logged actions in the persistent store and compare AURA's proposed replica trajectories against HPA's actual decisions using Grafana dashboards included in the repository. This removes the risk of premature MARL policy activation in production and provides an audit trail for policy evaluation over real traffic distributions before cut-over.

I. Limitations and Future Work

Cluster scale: k3d on a single host does not replicate the scheduling diversity and network latency variability of large multi-node clusters (EKS, GKE, AKS). Validation on managed cloud Kubernetes is a priority for future work.

Action space: AURA currently performs horizontal scaling only. Joint optimisation of replica count and resource requests (CPU/memory limits, VPA-style) would extend the action space to $\{-1, 0, +1\}^2$ per service and require retraining with a more complex reward function.

Distribution shift: The QMIX policy is trained on traffic distributions from Locust. Real-world traffic may exhibit patterns (e.g., diurnal periodicity, cascading failures) absent from the training distribution. Continual online fine-tuning or a meta-learning outer loop are natural extensions.

Scale to many services: The QMIX hypernetwork scales quadratically in the number of agents. For architectures with $n \gg 10$ microservices, hierarchical MARL [16] or attention-based mixing [17] offer more scalable alternatives with comparable coordination quality.

X. Conclusion

We presented **AURA**, an end-to-end MARL autoscaling framework that trains QMIX cooperative agents in a high-fidelity simulator and deploys them directly to a real Kubernetes cluster. By integrating with Prometheus, Envoy, and Locust, AURA bridges the long-standing gap between RL autoscaling research and practical cloud-native deployment.

The key innovation is a **16-dimensional predictive observation space** that includes queue depth, RPS derivatives, and CPU history— enabling proactive scaling decisions that reactive threshold-based policies cannot achieve. Production load tests on a 2-node k3d cluster demonstrated that QMIX achieves **46.37% better API P99 latency** (23.13 ms vs. 43.13 ms baseline) while using 50% more resources, and **4.32× better API latency than HPA** (23.13 ms vs. 99.87 ms) while using 55% less CPU (0.90 vs. 2.00 cores).

The results also revealed a controller-level bug: APP scale-up actions were suppressed by tier-coupled guard logic and then clamped by MIN_REPLICAS = 1, leaving APP fixed at one replica and causing elevated APP latency/error (P99 780.81 ms, 20.95%). This is an execution-path issue and not sufficient evidence of a QMIX architecture limitation.

Future directions include: (1) removing APP scale-up suppression in the controller and re-running full evaluations, (2) extending AURA to joint replica-and-resource optimization, (3) validating on large cloud-managed clusters, and (4) incorporating online continual fine-tuning to handle distribution shift. The complete codebase— including training code, Kubernetes manifests, simulator, and evaluation scripts—is available at https://github.com/Zane-Dev14/AURA.

Acknowledgment

The authors thank the open-source communities behind k3d, Prometheus, Envoy, Locust, and PettingZoo for providing the foundational tooling upon which AURA is built.

References

[1] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, vol. 80, pp. 4295–4304, 2018.

[2] F. Rossi, M. Nardelli, and V. Cardellini, "Horizontal and vertical scaling of container-based applications using reinforcement learning," in *Proc. IEEE Int. Conf. Cloud Comput. (CLOUD)*, pp. 329–338, 2019.

[3] L. Toka, G. Dobreff, B. Fodor, and B. Sonkoly, "Adaptive AI-based auto-scaling for Kubernetes," in *Proc. IEEE/ACM Int. Symp. Cluster Cloud Grid Comput. (CCGrid)*, pp. 599–608, 2021.

[4] M. Horovitz and Y. Arian, "SHOWAR: Right-sizing and efficient scheduling of microservices," in *Proc. ACM Symp. Cloud Comput. (SoCC)*, pp. 277–287, 2021.

[5] Kubernetes Authors, "Horizontal Pod Autoscaling," https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/, 2024.

[6] KEDA Authors, "Kubernetes Event-driven Autoscaling," https://keda.sh/, 2024.

[7] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at Google with Borg," in *Proc. 10th Eur. Conf. Comput. Syst. (EuroSys)*, pp. 18:1–18:17, 2015.

[8] K. Rzadca *et al.*, "Autopilot: Workload autoscaling at Google," in *Proc. 15th Eur. Conf. Comput. Syst. (EuroSys)*, pp. 1–16, 2020.

[9] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in *Proc. 15th ACM Workshop Hot Topics Netw. (HotNets)*, pp. 50–56, 2016.

[10] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, "Learning to route," in *Proc. 16th ACM Workshop Hot Topics Netw. (HotNets)*, pp. 1–7, 2017.

[11] T. Chu, S. Chinchali, and S. Katti, "Multi-agent deep reinforcement learning for large-scale traffic signal control," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 3, pp. 1086–1095, Mar. 2019.

[12] B. Kardatzki, F. Pallas, and P. Tuma, "Cooperative multi-agent autoscaling for heterogeneous cloud deployments," in *Proc. IEEE Int. Conf. Cloud Comput. (CLOUD)*, pp. 410–419, 2022.

[13] Z. Chen, Q. Wang, and S. Liu, "MARL-Kubernetes: Cooperative multi-agent reinforcement learning for microservice autoscaling," *J. Syst. Softw.*, vol. 198, p. 111590, 2023.

[14] J. K. Terry *et al.*, "PettingZoo: Gym for multi-agent reinforcement learning," in *Proc. 35th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2021.

[15] Locust Authors, "Locust: An open source load testing tool," https://locust.io/, 2024.

[16] S. Pateria, B. Subagdja, A.-H. Tan, and C. Quek, "Hierarchical reinforcement learning: A comprehensive survey," *ACM Comput. Surv.*, vol. 54, no. 5, pp. 109:1–109:35, Jun. 2021.

[17] S. Iqbal and F. Sha, "Actor-attention-critic for multi-agent reinforcement learning," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, pp. 2961–2970, 2019.